LAB 009

KUBERNETES ON EKS

INGRESS, SSL & PRODUCTION PATTERNS

EKS • Deployments • Services • Ingress • TLS • ConfigMaps • Secrets • HPA

Part 1: Kubernetes Concepts

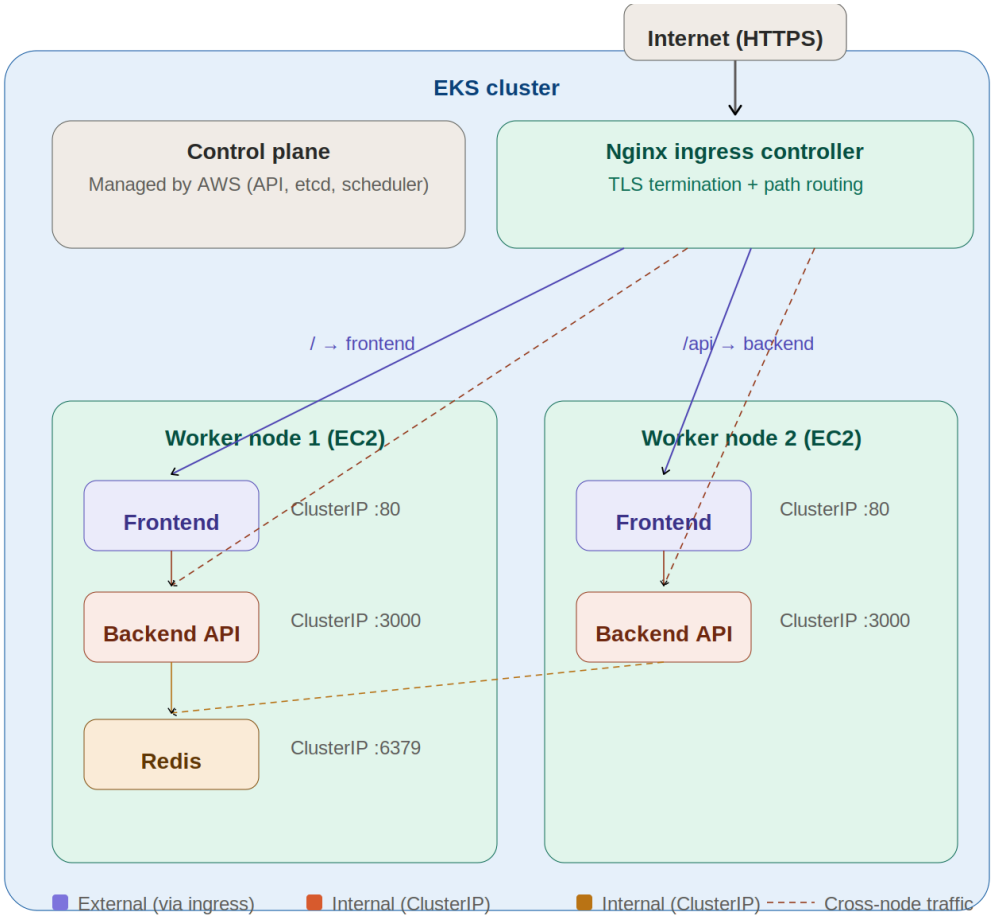
1.1 — What is Kubernetes?

In Lab 008, you used Docker Swarm to orchestrate containers across multiple machines. Kubernetes (K8s) does the same thing but at a much larger scale, with more features, and is the industry standard. Think of it as the operating system for your containers.

Kubernetes handles: scheduling containers on nodes, restarting crashed containers, scaling up/down based on load, load balancing traffic, rolling out updates with zero downtime, and managing configuration and secrets.

1.2 — Architecture

Here is the architecture you will build in this lab:



1.3 — Key Concepts

Concept	Explanation
Pod	The smallest deployable unit. Usually wraps 1 container. Pods are ephemeral — they can be killed and recreated at any time.
Deployment	Manages a set of identical Pods. You say "I want 3 replicas of my frontend" and the Deployment ensures 3 are always running.
Service	A stable network endpoint for a set of Pods. Pods have random IPs that change — a Service gives them a fixed DNS name.

ClusterIP Service	Internal only. Other pods inside the cluster can reach it, but external traffic cannot. Used for backend APIs and databases.
NodePort Service	Exposes a port on every node. External traffic can reach it via <NodeIP>:<Port>. Simple but not production-grade.
LoadBalancer Service	Creates an external load balancer (AWS ELB). Production-grade but creates one LB per service — expensive.
Ingress	A single entry point for external traffic. Routes requests to different services based on hostname or URL path. One LB for all services.
Ingress Controller	The actual proxy that implements Ingress rules. Nginx Ingress Controller is the most popular. It runs as pods inside your cluster.
ConfigMap	Stores non-sensitive configuration (env vars, config files) that pods can read. Like .env but managed by Kubernetes.
Secret	Like ConfigMap but for sensitive data (passwords, tokens). Base64 encoded. Mounted as env vars or files in pods.
Namespace	A virtual cluster inside your cluster. Used to isolate resources (e.g., dev vs staging vs production).
HPA	Horizontal Pod Autoscaler. Automatically scales pods based on CPU/memory usage. Similar to AWS Auto Scaling.

1.4 — Internal vs External Pods

A key concept in this lab is the distinction between public (external) and private (internal) services:

Type	How It Works
External (Public)	The frontend needs to be reachable from the internet. It gets traffic through the Ingress Controller. Users access it via a browser.
Internal (Private)	The backend API and Redis should NOT be directly reachable from the internet. They use ClusterIP services — only other pods in the cluster can talk to them.

This mirrors what you did in your AWS labs: frontend in public subnets, backend in private subnets. In Kubernetes, instead of subnets and security groups, you use Service types (ClusterIP vs LoadBalancer) and Ingress rules to control access.

Part 2: Setup — Install Tools & Create EKS Cluster

2.1 — Install kubectl

Step 1: Install kubectl (Kubernetes CLI)

```
# macOS
brew install kubectl

# Windows (chocolatey)
choco install kubernetes-cli

# Linux
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

kubectl version --client
```

2.2 — Install eksctl

Step 2: Install eksctl (EKS CLI)

```
# macOS
brew install eksctl

# Linux
curl --silent --location
"https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$(uname
-s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin

# Windows
choco install eksctl

eksctl version
```

2.3 — Create the EKS Cluster

Step 3: Create the Cluster with eksctl

```
eksctl create cluster \
  --name k8s-lab \
  --region us-east-1 \
  --nodegroup-name workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed
```

This takes 15–20 minutes. eksctl creates: a VPC with public/private subnets, an EKS control plane, an EC2 node group with 2 worker nodes, and all the IAM roles needed.

> ⚠️ **Warning**
> EKS costs ~$0.10/hr for the control plane + EC2 instance costs for worker nodes.
> t3.medium instances are NOT free tier. Delete the cluster as soon as you're done with the lab.

Step 4: Verify the Cluster

```
# kubectl should now be configured automatically
kubectl get nodes

# You should see 2 nodes in Ready status
kubectl cluster-info
```

Part 3: The Application — 3 Services

We'll deploy a 3-service application to demonstrate internal and external pods:

Service	Details
Frontend (External)	Nginx serving a static HTML page that calls the backend API. Accessible from the internet via Ingress.
Backend API (Internal)	Node.js Express API that reads/writes to Redis. Only accessible from within the cluster (ClusterIP).
Redis (Internal)	In-memory data store used by the backend. Only accessible from within the cluster (ClusterIP).

The Docker images are already built for you. We'll use public images to keep things simple:

```
# Frontend: Nginx with a custom HTML page
# Backend:  Node.js Express API with Redis
# Redis:    Official Redis image

# We'll create the frontend and backend images in the next steps
```

Step 5: Create the Project Structure

```
mkdir k8s-lab && cd k8s-lab
mkdir manifests app
```

Step 6: Create the Backend API (app/server.js)

```
const express = require("express");
const Redis = require("ioredis");
const app = express();
app.use(express.json());

const redis = new Redis({
  host: process.env.REDIS_HOST || 'localhost',
  port: 6379,
});

app.get("/api/health", async (req, res) => {
  const pong = await redis.ping();
  res.json({ status: "ok", redis: pong, pod: process.env.HOSTNAME });
});
```

```
app.get("/api/visits", async (req, res) => {
  const count = await redis.incr("visits");
  res.json({ visits: count, pod: process.env.HOSTNAME });
});

app.get("/api/messages", async (req, res) => {
  const msgs = await redis.lrange("messages", 0, -1);
  res.json(msgs.map(m => JSON.parse(m)));
});

app.post("/api/messages", async (req, res) => {
  const msg = { text: req.body.text, pod: process.env.HOSTNAME, time: new
Date().toISOString() };
  await redis.lpush("messages", JSON.stringify(msg));
  res.status(201).json(msg);
});

app.listen(3000, () => console.log('Backend running on :3000'));
```

Step 7: Create package.json and Dockerfile for Backend (app/)

app/package.json:

```
{ "name": "k8s-backend", "version": "1.0.0",
  "dependencies": { "express": "^4.18.0", "ioredis": "^5.3.0" },
  "scripts": { "start": "node server.js" } }
```

app/Dockerfile:

```
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install --production
COPY server.js .
EXPOSE 3000
CMD ["npm", "start"]
```

Step 8: Create the Frontend (app/frontend/)

```
mkdir -p app/frontend
```

app/frontend/index.html:

```
<!DOCTYPE html>
<html><head><title>K8s Lab</title>
```

```
<style>
  body { font-family: Arial; max-width: 700px; margin: 40px auto; padding:
20px; }
  .card { background: #f5f5f5; border-radius: 8px; padding: 16px; margin:
12px 0; }
  button { background: #326CE5; color: white; border: none; padding: 10px
20px;
           border-radius: 4px; cursor: pointer; font-size: 14px; }
  input { padding: 10px; width: 300px; border: 1px solid #ddd;
border-radius: 4px; }
</style></head><body>
  <h1>Kubernetes Lab — [YOUR NAME]</h1>
  <div class="card" id="health">Loading health...</div>
  <div class="card" id="visits">Loading visits...</div>
  <div class="card">
    <input id="msg" placeholder="Type a message">
    <button onclick="send()">Send</button>
  </div>
  <div id="messages"></div>
  <script>
    const api = '/api';
    fetch(api+'/health').then(r=>r.json()).then(d=>{
      document.getElementById('health').innerHTML=
        `<b>Health:</b> ${d.status} | <b>Redis:</b> ${d.redis} | <b>Pod:</b>
${d.pod}`;
    });
    fetch(api+'/visits').then(r=>r.json()).then(d=>{
      document.getElementById('visits').innerHTML=
        `<b>Total visits:</b> ${d.visits} | <b>Served by pod:</b> ${d.pod}`;
    });
    function load(){fetch(api+'/messages').then(r=>r.json()).then(msgs=>{
      document.getElementById('messages').innerHTML=msgs.map(m=>
        `<div class="card"><b>${m.pod}</b>: ${m.text}
<small>(${m.time})</small></div>`
      ).join('');
    });}
    function send(){

fetch(api+'/messages',{method:'POST',headers:{'Content-Type':'application/js
on'},

body:JSON.stringify({text:document.getElementById('msg').value})}).then(()=>
{
        document.getElementById('msg').value=''; load();
      });
    }
    load();
  </script>
</body></html>
```

⚠️ Warning

You MUST change [YOUR NAME] in index.html to your full name before building the Docker image. Your name must be visible in the browser for submission.

app/frontend/nginx.conf:

```
server {
    listen 80;
    root /usr/share/nginx/html;
    location / { try_files $uri $uri/ /index.html; }
    location /api/ {
        proxy_pass http://backend-service:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

app/frontend/Dockerfile:

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

Step 9: Build and Push Images to ECR

```
# Create ECR repos
aws ecr create-repository --repository-name k8s-frontend --region us-east-1
aws ecr create-repository --repository-name k8s-backend --region us-east-1

# Login to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin
<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com

# Build and push backend
cd app
docker build -t <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-backend:v1
.
docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-backend:v1

# Build and push frontend
cd frontend
docker build -t <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-frontend:v1
.
docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-frontend:v1
```

Replace <ACCOUNT_ID> with your AWS account ID throughout this lab.

Part 4: Kubernetes Manifests

Now we write the YAML manifests that tell Kubernetes what to deploy. Create each file in the manifests/ folder.

4.1 — Namespace

Step 10: Create Namespace (manifests/namespace.yaml)

```
apiVersion: v1
kind: Namespace
metadata:
  name: k8s-lab
```

4.2 — Redis (Internal)

Step 11: Redis Deployment + Service (manifests/redis.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: k8s-lab
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        ports:
        - containerPort: 6379
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: k8s-lab
spec:
```

```
  type: ClusterIP          # Internal only!
  selector:
    app: redis
  ports:
  - port: 6379
    targetPort: 6379
```

Notice `type: ClusterIP` — this means Redis is only reachable from inside the cluster. No external traffic can reach it.

4.3 — Backend API (Internal)

Step 12: Backend Deployment + Service (manifests/backend.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: k8s-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-backend:v1
        ports:
        - containerPort: 3000
        env:
        - name: REDIS_HOST
          value: redis-service     # K8s DNS name of the Redis service
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"
        livenessProbe:
          httpGet:
            path: /api/health
            port: 3000
          initialDelaySeconds: 10
```

```
          periodSeconds: 15
        readinessProbe:
          httpGet:
            path: /api/health
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: k8s-lab
spec:
  type: ClusterIP          # Internal only!
  selector:
    app: backend
  ports:
  - port: 3000
    targetPort: 3000
```

Liveness probe: K8s pings /api/health every 15s. If it fails 3 times, K8s kills and restarts the pod.

Readiness probe: K8s checks /api/health before sending traffic. If it fails, the pod is removed from the service until it recovers.

Resources: requests.cpu tells K8s how much CPU the pod needs. This is required for the Horizontal Pod Autoscaler (Part 8) to calculate CPU utilization. Without it, HPA shows <unknown> and never scales.

4.4 — Frontend (External via Ingress)

Step 13: Frontend Deployment + Service (manifests/frontend.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: k8s-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
```

```
      containers:
      - name: frontend
        image: <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/k8s-frontend:v1
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: k8s-lab
spec:
  type: ClusterIP          # Also ClusterIP! Ingress will handle external
access
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
```

Notice the frontend is also ClusterIP — not LoadBalancer. External traffic will come through the **Ingress Controller**, not directly to the service.

Step 14: Deploy Everything

```
kubectl apply -f manifests/namespace.yaml
kubectl apply -f manifests/redis.yaml
kubectl apply -f manifests/backend.yaml
kubectl apply -f manifests/frontend.yaml

# Check status
kubectl get all -n k8s-lab
```

```
# Wait for all pods to be ready
kubectl wait --for=condition=Ready pods --all -n k8s-lab --timeout=120s
```

You should see 5 pods running (1 Redis + 2 backend + 2 frontend) and 3 ClusterIP services. If any pod is stuck in Pending or CrashLoopBackOff, debug with:

```
kubectl describe pod <pod-name> -n k8s-lab
kubectl logs <pod-name> -n k8s-lab
```

Part 5: Nginx Ingress Controller

Right now, all services are internal (ClusterIP). We need an Ingress Controller to expose the frontend to the internet and route /api/* requests to the backend.

Step 15: Install Nginx Ingress Controller

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.4
/deploy/static/provider/aws/deploy.yaml

# Wait for it to be ready
kubectl get pods -n ingress-nginx --watch

# Get the external IP/hostname (AWS creates a Network Load Balancer)
kubectl get svc ingress-nginx-controller -n ingress-nginx
```

Copy the EXTERNAL-IP value — this is your NLB DNS name (e.g., abc123.elb.us-east-1.amazonaws.com). It may take 2–3 minutes to appear.

Step 16: Set Up nip.io Domain

nip.io is a free DNS service that maps any IP to a hostname. First, get your NLB's IP:

```
# Resolve the NLB DNS to get an IP
nslookup <NLB-DNS-NAME>

# Your nip.io domain will be:
# <IP>.nip.io
# Example: 54.210.33.100.nip.io
```

Now `54.210.33.100.nip.io` resolves to `54.210.33.100` — which is your Ingress Controller. Free, no registration needed.

> ⚠️ **Warning**
> NLBs can have multiple IPs that rotate. If your nip.io domain stops working, re-run nslookup to get the current IP and update your Ingress host and TLS cert. For a more stable setup, use the NLB DNS name directly with a curl --resolve override for testing.

Step 17: Create Ingress Resource (manifests/ingress.yaml)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```
metadata:
  name: app-ingress
  namespace: k8s-lab
spec:
  ingressClassName: nginx
  rules:
  - host: <YOUR-IP>.nip.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

Replace <YOUR-IP> with the NLB IP from the previous step.

```
kubectl apply -f manifests/ingress.yaml

# Test
curl http://<YOUR-IP>.nip.io/
curl http://<YOUR-IP>.nip.io/api/health
```

📝 **Note**

All traffic goes to the frontend service. The frontend's Nginx reverse proxy handles routing /api/* requests to the backend internally — the same pattern you used in Lab 007.

The Ingress only needs to know about the frontend. The backend stays fully internal — no direct external access.

Part 6: SSL/TLS with Self-Signed Certificate

HTTPS requires a TLS certificate. In production, you'd use a real cert from ACM or Let's Encrypt. For this lab, we'll create a self-signed certificate and configure the Ingress to terminate TLS.

Step 18: Generate a Self-Signed Certificate

```
# Generate a self-signed cert for your nip.io domain
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=<YOUR-IP>.nip.io"
```

Step 19: Create a K8s TLS Secret

```
kubectl create secret tls app-tls \
  --cert=tls.crt --key=tls.key \
  -n k8s-lab
```

Step 20: Update Ingress for TLS (manifests/ingress.yaml)

Add the tls section to your Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: k8s-lab
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - <YOUR-IP>.nip.io
    secretName: app-tls
  rules:
  - host: <YOUR-IP>.nip.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
```

```
            name: frontend-service
            port:
              number: 80
```

```
kubectl apply -f manifests/ingress.yaml

# Test HTTPS (use -k to skip cert verification for self-signed)
curl -k https://<YOUR-IP>.nip.io/api/health
```

Open https://<YOUR-IP>.nip.io in your browser. You'll see a warning (self-signed cert) — click through it. The page should load over HTTPS.

📝 **Note**

The ssl-redirect annotation forces all HTTP traffic to HTTPS. The Ingress Controller terminates TLS and forwards plain HTTP to the backend services.

Part 7: ConfigMaps & Secrets

Step 21: Create a ConfigMap (manifests/configmap.yaml)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: k8s-lab
data:
  APP_ENV: production
  LOG_LEVEL: info
  MAX_ITEMS: "100"
```

Then reference it in your backend deployment:

```
        envFrom:
        - configMapRef:
            name: app-config
```

This adds APP_ENV, LOG_LEVEL, and MAX_ITEMS as environment variables. Keep the existing REDIS_HOST env entry in your backend deployment — envFrom adds to it, it doesn't replace it.

Step 22: Create a Secret (manifests/secret.yaml)

```
# Base64 encode a value
echo -n 'MyS3cur3Token' | base64
# Output: TXlTM2N1cjNUb2tlbg==
```

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: k8s-lab
type: Opaque
data:
  API_TOKEN: TXlTM2N1cjNUb2tlbg==
```

Reference it in a deployment:

```
        env:
        - name: API_TOKEN
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: API_TOKEN
```

> ⚠️ **Warning**
> Secrets are base64 encoded, NOT encrypted. Try it yourself: run echo TXlTM2N1cjNUb2tlbg== | base64 -d and you'll see the original value in plain text.
>
> Anyone with cluster access can decode your secrets. In production, use AWS Secrets Manager with the CSI driver or external-secrets-operator.

Part 8: Scaling & Health Checks

8.1 — Manual Scaling

Step 23: Scale Up and Down

```
# Scale frontend to 4 replicas
kubectl scale deployment frontend --replicas=4 -n k8s-lab

# Watch pods come up
kubectl get pods -n k8s-lab -w

# Scale back to 2
kubectl scale deployment frontend --replicas=2 -n k8s-lab
```

8.2 — Horizontal Pod Autoscaler (HPA)

Step 24: Create HPA for Backend

First, install metrics-server (needed for HPA):

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/c
omponents.yaml
```

If metrics-server pods crash with certificate errors, patch them:

```
kubectl patch deployment metrics-server -n kube-system --type='json' \

-p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"-
-kubelet-insecure-tls"}]'
```

Then create the HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: k8s-lab
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 6
  metrics:
```

```
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

This auto-scales the backend between 2 and 6 pods based on CPU usage. Similar to AWS Auto Scaling policies from Lab 006.

```
kubectl get hpa -n k8s-lab
```

8.3 — Pod Recovery Demo

Step 25: Kill a Pod and Watch Recovery

```
# List pods
kubectl get pods -n k8s-lab

# Delete a backend pod
kubectl delete pod <BACKEND-POD-NAME> -n k8s-lab

# Watch K8s recreate it immediately
kubectl get pods -n k8s-lab -w
```

The Deployment controller notices a pod is missing and creates a new one within seconds. This is self-healing.

Part 9: ALB Ingress Controller (Bonus / Optional)

The Nginx Ingress Controller creates a Network Load Balancer (NLB). AWS also offers the AWS Load Balancer Controller which creates Application Load Balancers (ALBs) — the same ALBs you used in your AWS labs. This section is optional and provided as a reference for students who want to explore the AWS-native approach.

9.1 — Install AWS Load Balancer Controller

Follow the official AWS docs to install:
https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html

Key steps:

- Create an IAM policy for the controller
- Create an IAM service account using eksctl
- Install the controller via Helm

9.2 — ALB Ingress Resource

An ALB Ingress looks different — it uses annotations to configure the ALB:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress-alb
  namespace: k8s-lab
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: <ACM-CERT-ARN>
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

> 📝 **Note**
> Like our Nginx Ingress setup, this routes all traffic to the frontend. The frontend's Nginx reverse proxy handles /api/* routing to the backend internally.
>
> The certificate-arn annotation attaches an ACM certificate to the ALB. ACM certs are free but require a real domain with DNS validation.

9.3 — Comparison: Nginx Ingress vs ALB Ingress

Aspect	Nginx Ingress vs ALB Ingress
Load Balancer	Nginx: creates a NLB. ALB: creates an ALB.
SSL	Nginx: you manage certs (self-signed, cert-manager). ALB: ACM certs (free, auto-renewed).
Cost	Nginx: 1 NLB for all services. ALB: 1 ALB per Ingress resource.
Portability	Nginx: works on any K8s cluster (EKS, GKE, on-prem). ALB: AWS only.
Features	Nginx: rate limiting, rewrites, auth. ALB: WAF, access logs, built-in DDoS protection.

Part 10: kubectl Quick Reference

Here are the kubectl commands you'll use most often. Bookmark this page — you'll come back to it constantly.

10.1 — Cluster & Context

Command	What It Does
kubectl cluster-info	Show the cluster API server URL and CoreDNS address.
kubectl get nodes	List all worker nodes and their status (Ready/NotReady).
kubectl get nodes -o wide	Same as above but with IP addresses, OS, and container runtime.
kubectl config current-context	Show which cluster kubectl is currently talking to.
kubectl config get-contexts	List all configured clusters/contexts.
kubectl config use-context <name>	Switch to a different cluster context.

10.2 — Viewing Resources

Command	What It Does
kubectl get pods -n <ns>	List all pods in a namespace. Add -o wide for IPs and node placement.
kubectl get all -n <ns>	List pods, services, deployments, replicasets — everything in a namespace.
kubectl get svc -n <ns>	List all services (shows ClusterIP, NodePort, LoadBalancer types).
kubectl get deploy -n <ns>	List deployments with desired/current/ready replica counts.
kubectl get ingress -n <ns>	List Ingress resources with hosts, paths, and backends.
kubectl get events -n <ns> --sort-by='.lastTimestamp'	Show recent cluster events (useful for debugging).
kubectl get pvc -n <ns>	List PersistentVolumeClaims and their status (Bound/Pending).
kubectl get secrets -n <ns>	List secrets (names and types, not values).

kubectl get configmaps -n <ns>	List ConfigMaps.
kubectl get hpa -n <ns>	Show Horizontal Pod Autoscaler status and current metrics.

10.3 — Inspecting & Debugging

Command	What It Does
kubectl describe pod <name> -n <ns>	Detailed info: events, conditions, container status, volumes. First thing to run when a pod won't start.
kubectl logs <pod> -n <ns>	View stdout/stderr from a pod. Add -f to follow (like tail -f).
kubectl logs <pod> -c <container> -n <ns>	View logs from a specific container (if pod has multiple containers).
kubectl logs <pod> --previous -n <ns>	View logs from the PREVIOUS crashed container (before K8s restarted it).
kubectl exec -it <pod> -n <ns> -- /bin/sh	Open a shell inside a running pod (like docker exec). Use bash or sh.
kubectl exec -it <pod> -n <ns> -- curl http://backend-service:3000 /api/health	Run a one-off command inside a pod (test internal connectivity).
kubectl port-forward svc/<svc> 8080:80 -n <ns>	Forward a local port to a service. Access via localhost:8080. Great for testing without Ingress.
kubectl top pods -n <ns>	Show CPU and memory usage per pod (requires metrics-server).
kubectl top nodes	Show CPU and memory usage per node.

10.4 — Creating & Modifying

Command	What It Does
kubectl apply -f <file.yaml>	Create or update resources from a YAML file. Idempotent — safe to run multiple times.
kubectl apply -f manifests/	Apply ALL YAML files in a directory at once.
kubectl delete -f <file.yaml>	Delete the resources defined in a YAML file.
kubectl delete pod <name> -n <ns>	Delete a specific pod (Deployment will recreate it).

kubectl scale deploy <name> --replicas=4 -n <ns>	Scale a deployment to 4 replicas.
kubectl rollout restart deploy <name> -n <ns>	Restart all pods in a deployment (pulls fresh image if tag changed).
kubectl rollout status deploy <name> -n <ns>	Watch a rolling update in progress.
kubectl rollout undo deploy <name> -n <ns>	Roll back to the previous deployment version.
kubectl set image deploy/<name> <container>=<image:tag> -n <ns>	Update the image of a deployment (triggers rolling update).

10.5 — Namespaces

Command	What It Does
kubectl get namespaces	List all namespaces.
kubectl create namespace <name>	Create a new namespace.
kubectl delete namespace <name>	Delete a namespace and ALL resources inside it.
kubectl config set-context --current --namespace=<ns>	Set a default namespace so you don't need -n every time.

✅ **Tip**

Almost every command accepts -n <namespace>. If you're tired of typing -n k8s-lab on every command, set it as your default with the set-context command above.

Add -o yaml to any get command to see the full YAML definition of a resource. Add -o json for JSON.

Part 11: Jumphost — Accessing the Cluster from EC2

So far you've been running kubectl from your local machine. In production, you often manage clusters from a jumphost (bastion) inside the VPC. Let's set one up.

11.1 — Why a Jumphost?

- **Security:** Your EKS API endpoint can be made private — only reachable from inside the VPC. A jumphost lets you reach it without exposing the API to the internet.
- **Consistency:** Everyone on the team uses the same environment with the same tools installed.
- **Auditing:** All cluster access goes through one machine that you can monitor and log.

11.2 — Launch a Jumphost EC2

Step 27: Launch the Jumphost

Launch a t2.micro EC2 instance (Amazon Linux 2023) in one of the public subnets that eksctl created. Use the same VPC as the EKS cluster.

Security group rules:

Inbound: SSH (22) from your IP

Outbound: All traffic

Make sure to select a key pair you have access to.

11.3 — Install Tools on the Jumphost

Step 28: SSH In and Install kubectl + AWS CLI

```
# SSH into the jumphost
ssh -i your-key.pem ec2-user@<JUMPHOST-PUBLIC-IP>

# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/
kubectl version --client
```

```
# AWS CLI is pre-installed on Amazon Linux 2023
aws --version
```

11.4 — Configure AWS Credentials on the Jumphost

Step 29: Configure Credentials

Option A: Run aws configure and enter your access keys (simple but less secure):

```
aws configure
```

Option B (recommended): Attach an IAM instance profile to the jumphost with EKS permissions. No credentials stored on disk.

```
# Create a policy that allows EKS access
# Attach it to a role, then attach the role to the EC2 instance
# Required permissions: eks:DescribeCluster, eks:ListClusters
```

11.5 — Connect kubectl to the EKS Cluster

Step 30: Update kubeconfig

```
# This configures kubectl to talk to your EKS cluster
aws eks update-kubeconfig --name k8s-lab --region us-east-1

# Verify
kubectl get nodes
kubectl get pods -n k8s-lab
```

You should see the same nodes and pods as from your local machine.

11.6 — Test Internal Connectivity

Step 31: Verify Access to Internal Services

From the jumphost, you can test internal cluster networking:

```
# Port-forward a ClusterIP service to the jumphost
kubectl port-forward svc/backend-service 8080:3000 -n k8s-lab &

# Now test the backend API (only reachable internally)
curl http://localhost:8080/api/health
curl http://localhost:8080/api/visits

# Stop the port-forward
kill %1
```

This is how you debug internal services without exposing them to the internet. Port-forward creates a tunnel from the jumphost to the pod.

Step 32: Run a Debug Pod Inside the Cluster

For deeper debugging, spin up a temporary pod inside the cluster:

```
# Launch a temporary debug pod with curl installed
kubectl run debug --rm -it --image=curlimages/curl -n k8s-lab -- sh

# Inside the debug pod, test internal DNS and connectivity:
curl http://backend-service:3000/api/health
curl http://redis-service:6379
curl http://frontend-service:80

# Test DNS resolution
nslookup backend-service.k8s-lab.svc.cluster.local

# Exit (pod auto-deletes because of --rm)
exit
```

This is the ultimate debugging tool. The debug pod runs **inside the cluster network** — it can reach every ClusterIP service by DNS name, exactly like your application pods do.

> 📝 **Note**
> The --rm flag auto-deletes the debug pod when you exit. The -it flags give you an interactive terminal.
> This is similar to SSHing into a bastion host in your AWS labs — but instead of jumping into a VPC, you're jumping into the Kubernetes network.

11.7 — Jumphost Cheat Sheet

Task	Command from Jumphost
See all pods	kubectl get pods -n k8s-lab -o wide

Check pod logs	kubectl logs <pod-name> -n k8s-lab -f
Shell into a pod	kubectl exec -it <pod-name> -n k8s-lab -- /bin/sh
Test internal service	kubectl port-forward svc/<name> 8080:<port> -n k8s-lab
Debug from inside cluster	kubectl run debug --rm -it --image=curlimages/curl -n k8s-lab -- sh
Watch pods in real-time	kubectl get pods -n k8s-lab -w
Check resource usage	kubectl top pods -n k8s-lab
View cluster events	kubectl get events -n k8s-lab --sort-by='.lastTimestamp'

> ⚠️ **Warning**
> Don't forget to terminate the jumphost EC2 instance when you're done with the lab. It's a t2.micro so it's free-tier eligible, but clean up anyway.

Part 12: Submission

Before taking screenshots, make sure your name is visible on the frontend page.

Screenshot 1: Terminal showing kubectl get all -n k8s-lab with all pods Running, services, and deployments.

Screenshot 2: Terminal showing kubectl get nodes -o wide with both worker nodes in Ready state.

Screenshot 3: Browser showing the app via the nip.io HTTPS URL. Your name must be visible. The health check card should show the backend pod name.

Screenshot 4: Browser refreshed showing a different backend pod name (proving load balancing across replicas).

Screenshot 5: Terminal showing kubectl get ingress -n k8s-lab with the host, TLS, and backend configured.

Screenshot 6: Terminal showing curl -k https://<YOUR-IP>.nip.io/api/health returning JSON with status and pod name.

Screenshot 7: Terminal showing kubectl scale deployment frontend --replicas=4 -n k8s-lab followed by kubectl get pods -n k8s-lab showing 4 frontend pods.

Screenshot 8: Terminal showing kubectl delete pod <backend-pod> -n k8s-lab followed by kubectl get pods showing K8s recreating it automatically.

Screenshot 9: Terminal showing kubectl get hpa -n k8s-lab with the backend HPA configured.

Screenshot 10: Terminal from the jumphost showing kubectl get pods -n k8s-lab and a successful curl to the backend via port-forward.

Screenshot 11: Terminal showing eksctl delete cluster completing successfully (take this LAST, after all other screenshots).

Submit: Combine all screenshots into a single PDF and submit through the link on Moodle.

Part 13: Clean Up

> ⚠️ **Warning**
> EKS charges ~$0.10/hr for the control plane + EC2 costs for worker nodes.
> DELETE THE CLUSTER as soon as you're done. Do not leave it running overnight.

Step 33: Delete Everything

```
# Delete all resources in the namespace
kubectl delete namespace k8s-lab

# Delete the Ingress Controller (this also deletes the NLB)
kubectl delete -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.4
/deploy/static/provider/aws/deploy.yaml

# Delete the EKS cluster (takes ~10 minutes)
eksctl delete cluster --name k8s-lab --region us-east-1

# Delete ECR repositories
aws ecr delete-repository --repository-name k8s-frontend --force --region
us-east-1
aws ecr delete-repository --repository-name k8s-backend --force --region
us-east-1

# Terminate the jumphost EC2 instance
# Go to AWS Console → EC2 → Instances → select jumphost → Terminate
```

Verify in the AWS Console: no EKS clusters, no EC2 instances (including jumphost), no load balancers, no ECR repos.

Lab Recap

What You Did	K8s Concepts
Created an EKS cluster	Managed control plane, worker nodes, eksctl
Deployed 3 services	Deployments, Pods, ReplicaSets
Internal vs External	ClusterIP (Redis, Backend) vs Ingress (Frontend)
Nginx Ingress Controller	Path-based routing, single entry point
SSL/TLS with nip.io	Self-signed certs, TLS termination at Ingress
ConfigMaps & Secrets	Externalized config, base64-encoded secrets

Scaling & HPA	Manual scaling, auto-scaling on CPU
Health checks	Liveness probes (restart) + Readiness probes (traffic)
ALB Ingress (alternative)	AWS-native: ACM certs + ALB via annotations
Pod recovery	Self-healing: delete a pod, K8s recreates it
kubectl reference	get, describe, logs, exec, port-forward, scale, rollout
Jumphost access	EC2 bastion for cluster management, debug pods, port-forwarding

---

End of Lab 009 — You can now deploy, expose, secure, and scale apps on Kubernetes.